

E-Commerce Database Management System using SQL

Submitted by – Soumil Manhas

Role Applied – Java Full Stack Engineer

1. Problem Statement :

The objective of this project is to design and manage a relational database system for an e-commerce platform using SQL. The project demonstrates table creation, data insertion, joins, aggregations, subqueries, and analytical queries used in backend systems.

2. Tools Used : MySQL Workbench

3. Database Schema :

- **Customers Table** : Stores customer details such as customer ID, name, and city.

```
CREATE TABLE Customers (  
    customer_id INT PRIMARY KEY,  
    customer_name VARCHAR(100),  
    city VARCHAR(50));
```

INSERT INTO Customers VALUES

(1, 'Rahul', 'Delhi'),
(2, 'Aman', 'Mumbai'),
(3, 'Sneha', 'Bangalore'),
(4, 'Priya', 'Hyderabad');

50 • `SELECT * FROM Customers;`

Result Grid | Filter Rows: | Edit: | Export/Import:

	customer_id	customer_name	city
▶	1	Rahul	Delhi
	2	Aman	Mumbai
	3	Sneha	Bangalore
	4	Priya	Hyderabad
*	NULL	NULL	NULL

- **Products Table** : Stores product information including category and price.

CREATE TABLE Products (
 product_id INT PRIMARY KEY,
 product_name VARCHAR(100),
 category VARCHAR(50),
 price INT);

INSERT INTO Products VALUES

(101, 'Laptop', 'Electronics', 55000),
(102, 'Mouse', 'Electronics', 700),
(103, 'Keyboard', 'Electronics', 1200),
(104, 'Smartphone', 'Electronics', 30000);

52 • `SELECT * FROM Products;`

Result Grid

	product_id	product_name	category	price
▶	101	Laptop	Electronics	55000
	102	Mouse	Electronics	700
	103	Keyboard	Electronics	1200
	104	Smartphone	Smartphone	30000
•	NULL	NULL	NULL	NULL

Customers 11 Customers 13 Products 14 x Apply Revert

Output

Action Output

#	Time	Action	Message
68	14:50:20	CREATE TABLE Products (product_id INT PRIMARY KEY, product_name VARCHAR(100), category ...	0 row(s) affected

- **Orders Table** : Stores order transactions linked with customers.

```
CREATE TABLE Orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    order_date DATE,
    total_amount INT,
    FOREIGN KEY (customer_id) REFERENCES
Customers(customer_id)
);
```

```
INSERT INTO Orders VALUES
(1001, 1, '2026-05-10', 55700),
(1002, 2, '2026-05-11', 30000),
(1003, 3, '2026-05-12', 1200),
(1004, 1, '2026-05-15', 700);
```

54 • `SELECT * FROM Orders;`

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

	order_id	customer_id	order_date	total_amount
▶	1001	1	2026-05-10	55700
	1002	2	2026-05-11	30000
	1003	3	2026-05-12	1200
	1004	1	2026-05-15	700
•	NULL	NULL	NULL	NULL

Customers 11 Customers 15 Products 16 Orders 17 × Apply Revert

Result Grid
Form Editor
Field Types

4. SQL Queries

- **SELECT Queries**
 - i) `SELECT * FROM Customers;`
 - ii) `SELECT * FROM Products;`
 - iii) `SELECT * FROM Orders;`
- **INNER JOIN Query**

```
SELECT c.customer_name,
       o.order_id,
       o.total_amount
FROM Customers c
INNER JOIN Orders o
ON c.customer_id = o.customer_id;
```

Output:

```
56 • SELECT c.customer_name,  
57         o.order_id,  
58         o.total_amount  
59 FROM Customers c  
60 INNER JOIN Orders o  
61 ON c.customer_id = o.customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	customer_name	order_id	total_amount
▶	Rahul	1001	55700
	Rahul	1004	700
	Aman	1002	30000
	Sneha	1003	1200

Customers 11 Customers 18 Products 19 Orders 20 Result 21 x Read Only

Result Grid
Form Editor
Field Types

- **GROUP BY Query**

```
SELECT city,  
        COUNT(*) AS total_customers  
FROM Customers  
GROUP BY city;
```

Output :

```
63 • SELECT city,  
64         COUNT(*) AS total_customers  
65 FROM Customers  
66 GROUP BY city;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

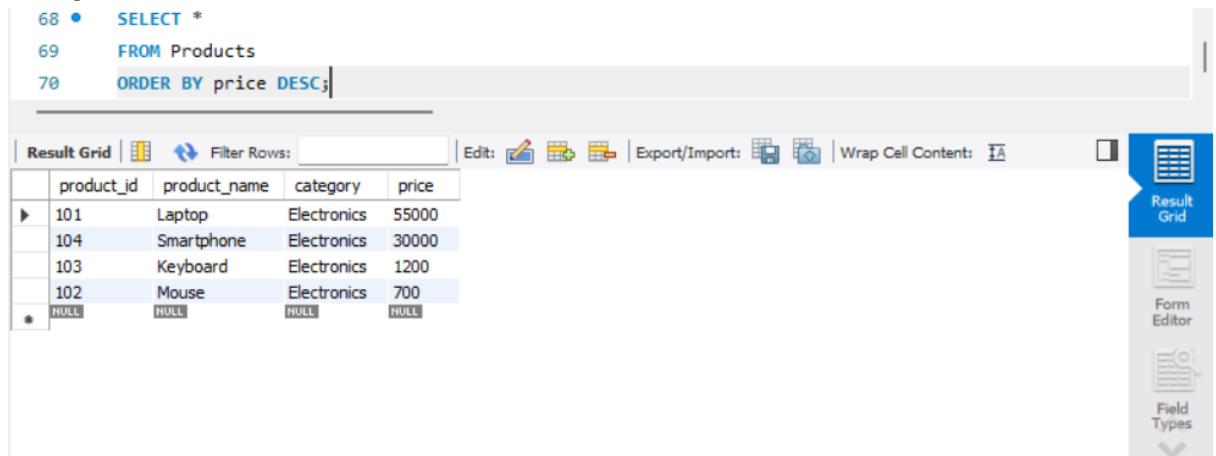
	city	total_customers
▶	Delhi	1
	Mumbai	1
	Bangalore	1
	Hyderabad	1

Result Grid
Form Editor
Field Types

- **ORDER BY Query**

```
SELECT *
FROM Products
ORDER BY price DESC;
```

Output:



The screenshot shows a database query editor with the following SQL query:

```
68 • SELECT *
69 FROM Products
70 ORDER BY price DESC;
```

Below the query editor is a toolbar with options like 'Result Grid', 'Filter Rows', 'Edit', 'Export/Import', and 'Wrap Cell Content'. The 'Result Grid' is active, displaying the following data:

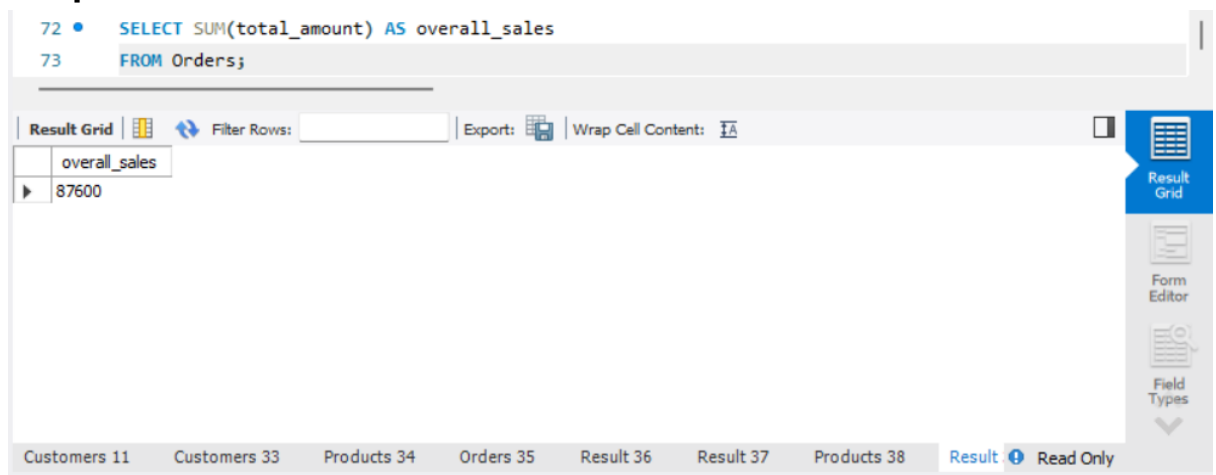
	product_id	product_name	category	price
▶	101	Laptop	Electronics	55000
	104	Smartphone	Electronics	30000
	103	Keyboard	Electronics	1200
	102	Mouse	Electronics	700
*	NULL	NULL	NULL	NULL

On the right side of the interface, there are buttons for 'Result Grid', 'Form Editor', and 'Field Types'.

- **Aggregate Function**

```
SELECT SUM(total_amount) AS overall_sales
FROM Orders;
```

Output:



The screenshot shows a database query editor with the following SQL query:

```
72 • SELECT SUM(total_amount) AS overall_sales
73 FROM Orders;
```

Below the query editor is a toolbar with options like 'Result Grid', 'Filter Rows', 'Export', and 'Wrap Cell Content'. The 'Result Grid' is active, displaying the following data:

	overall_sales
▶	87600

At the bottom of the interface, there is a tab bar with the following tabs: 'Customers 11', 'Customers 33', 'Products 34', 'Orders 35', 'Result 36', 'Result 37', 'Products 38', 'Result', and 'Read Only'.

- **Top Customers by Total Spending**

```
SELECT c.customer_name,
```

```
SUM(o.total_amount) AS total_spending
FROM Customers c
JOIN Orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_name
ORDER BY total_spending DESC;
```

Output:

```
75 • SELECT c.customer_name,
76       SUM(o.total_amount) AS total_spending
77 FROM Customers c
78 JOIN Orders o
79 ON c.customer_id = o.customer_id
80 GROUP BY c.customer_name
81 ORDER BY total_spending DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	customer_name	total_spending
▶	Rahul	56400
	Aman	30000
	Sneha	1200

Customers 11 Customers 40 Products 41 Orders 42 Result 43 Result 44 Products 45 Result 47 Read Only 47 ×

Result Grid
Form Editor
Field Types

- Subquery

```
SELECT customer_name
FROM Customers
WHERE customer_id IN (
    SELECT customer_id
    FROM Orders
    WHERE total_amount > 30000);
```

Output:

```
83 • SELECT customer_name
84 FROM Customers
85 WHERE customer_id IN (
86     SELECT customer_id
87     FROM Orders
88     WHERE total_amount > 30000
89 );
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

customer_name
Rahul

Customers 48 | Products 49 | Orders 50 | Result 51 | Result 52 | Products 53 | Result 54 | Result 55 | Read Only x

- **CREATE View**

```
CREATE VIEW High_Value_Orders AS
SELECT *
FROM Orders
WHERE total_amount > 30000;
```

Output:

```
91 • CREATE VIEW High_Value_Orders AS
92 SELECT *
93 FROM Orders
94 WHERE total_amount > 30000;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

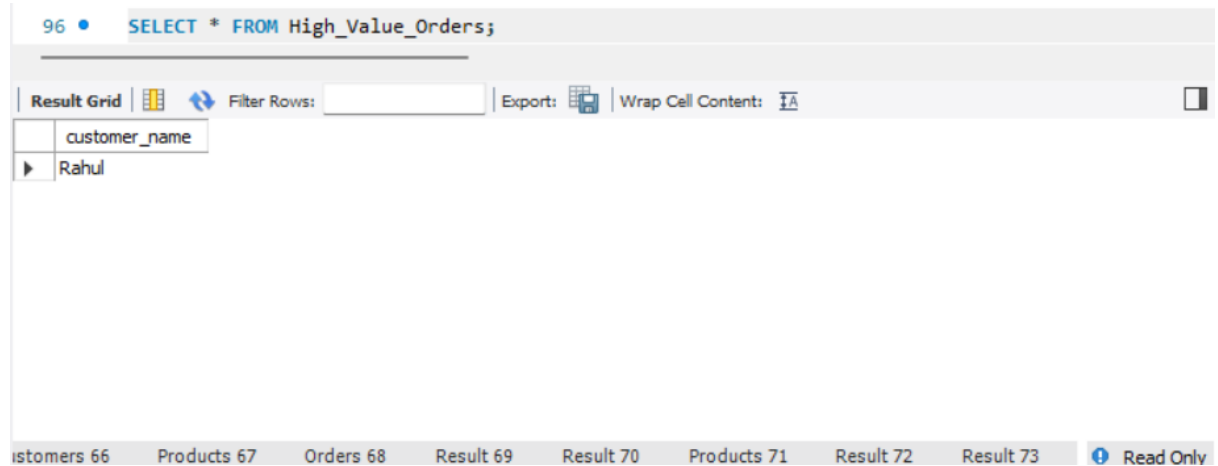
customer_name
Rahul

Customers 48 | Products 49 | Orders 50 | Result 51 | Result 52 | Products 53 | Result 54 | Result 55 | Read Only x

- **VIEW Query**

SELECT * FROM High_Value_Orders;

Output:



customer_name
Rahul

4. Key Learnings:

- **Learned how to design and manage relational databases using SQL.**
- **Gained practical experience in creating tables, defining primary and foreign keys, and maintaining relationships between multiple tables.**
- **Implemented SQL operations including SELECT, INSERT, JOIN, GROUP BY, ORDER BY, aggregate functions, subqueries, and views.**
- **Improved understanding of how backend systems use databases to store, retrieve, and analyze business data efficiently.**
- **Developed analytical thinking by solving real-world business scenarios such as customer spending analysis and sales tracking.**
- **Enhanced query-writing skills and learned how to generate meaningful insights from structured datasets.**